



Universidad Nacional del Nordeste



Facultad de Ciencias Exactas y Naturales y Agrimensura

Cátedra: Sistemas Operativos

Año: 2026

Trabajos Práctico 1. Procesos y administrador del
Procesador.

Alumno: Aranda Gabriel Angel

D.N.I. N°: 46714612

Resolución del Práctico N° 1: Procesos y administración del procesador

Ejercicio 1:

Proceso	Tiempo de Llegada	Ráfaga de CPU
P1	0	7
P2	2	5
P3	3	2
P4	5	4

1.1. FCFS

FCFS ejecuta los procesos en el orden en que llegan.

Diagrama de Gantt:

0 — P1 — 7 — P2 — 12 — P3 — 14 — P4 — 18

Proceso	Finalización	Retorno	Espera
P1	7	$7 - 0 = 7$	$7 - 7 = 0$
P2	12	$12 - 2 = 10$	$10 - 5 = 5$
P3	14	$14 - 3 = 11$	$11 - 2 = 9$
P4	18	$18 - 5 = 13$	$13 - 4 = 9$

Tiempo de espera promedio: 5,75 ms.

Tiempo de retorno promedio: 10,25 ms.

1.2. SJF no apropiativo

SJF selecciona, entre los procesos disponibles, el de menor ráfaga.

Diagrama de Gantt:

0 — P1 — 7 — P3 — 9 — P4 — 13 — P2 — 18

Proceso	Finalización	Retorno	Espera
P1	7	$7 - 0 = 7$	$7 - 7 = 0$
P2	18	$18 - 2 = 16$	$16 - 5 = 11$
P3	9	$9 - 3 = 6$	$6 - 2 = 4$
P4	13	$13 - 5 = 8$	$8 - 4 = 4$

Tiempo de espera promedio: 4,75 ms.

Tiempo de retorno promedio: 9,27 ms.

1.3. SRT apropiativo

SRT selecciona siempre el proceso con menor tiempo restante de CPU. Se adopta la regla de que un proceso en ejecución no es desalojado si el nuevo proceso tiene exactamente el mismo tiempo restante.

Diagrama de Gantt:

0 — P1 — 3 — P3 — 5 — P1 — 9 — P4 — 13 — P2 — 18

Proceso	Finalización	Retorno	Espera
P1	9	$9 - 0 = 9$	$9 - 7 = 2$
P2	18	$18 - 2 = 16$	$16 - 5 = 11$
P3	5	$5 - 3 = 2$	$2 - 2 = 0$
P4	13	$13 - 5 = 8$	$8 - 4 = 4$

Tiempo de espera promedio: 4,25 ms.

Tiempo de retorno promedio: 8,75 ms.

1.4. Round Robin con cuanto de cuatro milisegundos

Se utiliza un cuanto de cuatro milisegundos. Los procesos que llegan durante un intervalo se incorporan a la cola antes de reencolar el proceso cuyo cuanto finaliza.

Diagrama de Gantt:

0 — P1 — 4 — P2 — 8 — P3 — 10 — P1 — 14 — P4 — 18 — P2 — 19

Proceso	Finalización	Retorno	Espera
P1	14	$14 - 0 = 14$	$14 - 7 = 7$
P2	19	$19 - 2 = 17$	$17 - 5 = 12$
P3	10	$10 - 3 = 7$	$7 - 2 = 5$
P4	18	$18 - 5 = 13$	$13 - 4 = 9$

Tiempo de espera promedio: 7,5 ms.

Tiempo de retorno promedio: 12 ms.

Ejercicio 2:

Proceso	Llegada	Ráfaga	Prioridad
P1	0	9	2
P2	3	5	4
P3	5	6	1
P4	7	1	3
P5	8	6	1

2.1. Prioridades no apropiativo

En la planificación no apropiativa, una vez que un proceso obtiene la CPU continúa hasta terminar su ráfaga.

Diagrama de Gantt:

0 — P1 — 9 — P3 — 15 — P5 — 21 — P4 — 22 — P2 — 27

Secuencia de ejecución: P1, P3, P5, P4, P2.

Proceso	Finalización	Retorno	Espera
P1	9	9	0
P2	27	$27 - 3 = 24$	$24 - 5 = 19$
P3	15	$15 - 5 = 10$	$10 - 6 = 4$

P4	22	$22 - 7 = 15$	$15 - 1 = 14$
P5	21	$21 - 8 = 13$	$13 - 6 = 7$

Tiempo de espera promedio: 8,8 ms.

Tiempo de retorno promedio: 14,2 m

2.2. Prioridades apropiativo

En la planificación apropiativa, la llegada de un proceso con prioridad superior puede desalojar al proceso que está utilizando la CPU.

Diagrama de Gantt:

0 — P1 — 5 — P3 — 11 — P5 — 17 — P1 — 21 — P4 — 22 — P2 — 27

Secuencia de ejecución: P1, P3, P5, P1, P4, P2.

Proceso	Finalización	Retorno	Espera
P1	21	$21 - 0 = 21$	$21 - 9 = 12$
P2	27	$27 - 3 = 24$	$24 - 5 = 19$
P3	11	$11 - 5 = 6$	$6 - 6 = 0$
P4	22	$22 - 7 = 15$	$15 - 1 = 14$
P5	17	$17 - 8 = 9$	$9 - 6 = 3$

Tiempo de espera promedio: 9,6 ms.

Tiempo de retorno promedio: 15 ms.

2.3. Proceso con mayor riesgo de inanición

En este conjunto, el proceso que presenta mayor riesgo de inanición es P2, porque posee la prioridad más baja: prioridad 4. En el esquema no apropiativo no es desalojado durante la ejecución de P1, pero luego debe esperar mientras se ejecutan P3, P5 y P4, todos con prioridad superior.

En el ejemplo concreto P2 finalmente obtiene la CPU, por lo que no existe espera indefinida real. Sin embargo, si continuaran llegando procesos con prioridades 1, 2 o 3, P2 podría permanecer indefinidamente en la cola de listos. Esto constituye inanición o *starvation*: el proceso está listo para ejecutarse, pero la política de asignación de CPU favorece repetidamente a otros procesos.

Ejercicio 3:

Proceso	Llegada	Ráfaga
A	0	5
B	3	4
C	4	3
D	6	2

3.1. Round Robin con cuanto de dos milisegundos

Se incorporan los procesos que llegan antes de reencolar el proceso que termina su cuanto.

Diagrama de Gantt:

0 — A — 2 — A — 4 — B — 6 — C — 8 — A — 9 — D — 11 — B — 13 — C — 14

Proceso	Finalización	Retorno	Espera
A	9	$9 - 0 = 9$	$9 - 5 = 4$
B	13	$13 - 3 = 10$	$10 - 4 = 6$
C	14	$14 - 4 = 10$	$10 - 3 = 7$
D	11	$11 - 6 = 5$	$5 - 2 = 3$

Tiempo de espera promedio: 5 ms.

3.2. Round Robin con cuanto de cuatro milisegundos

Diagrama de Gantt:

0 — A — 4 — B — 8 — C — 11 — A — 12 — D — 14

Proceso	Finalización	Retorno	Espera
A	12	$12 - 0 = 12$	$12 - 5 = 7$

B	8	$8 - 3 = 5$	$5 - 4 = 1$
C	11	$11 - 4 = 7$	$7 - 3 = 4$
D	14	$14 - 6 = 8$	$8 - 2 = 6$

Tiempo de espera promedio: 4,5 ms.

3.3. Efecto del tamaño del cuanto

Con un cuanto de dos milisegundos, los procesos se intercalan con mayor frecuencia. Esto favorece la percepción de respuesta rápida, especialmente en sistemas interactivos, porque ningún proceso debe esperar demasiado tiempo para volver a utilizar la CPU.

Sin embargo, un cuanto pequeño produce más cambios de contexto. Cada cambio implica guardar el estado del proceso saliente, cargar el estado del proceso entrante y actualizar las estructuras del sistema operativo. Ese trabajo no ejecuta instrucciones útiles del usuario y constituye sobrecarga.

Con un cuanto de cuatro milisegundos, se realizan menos alternancias. Por ello disminuye la sobrecarga por cambios de contexto y, en este conjunto, también se obtiene un menor tiempo de espera promedio. La contrapartida es que un proceso puede conservar la CPU durante un intervalo mayor, lo que puede aumentar el tiempo de respuesta de otros procesos recién llegados.

En consecuencia, el cuanto debe equilibrar dos objetivos:

- Un cuanto demasiado pequeño mejora la interactividad, pero aumenta la cantidad de cambios de contexto.
- Un cuanto demasiado grande reduce la sobrecarga, pero puede hacer que Round Robin se comporte de manera semejante a FCFS y empeore la respuesta interactiva.

Ejercicio 4:

Se consideran los estados Nuevo, Listo, Ejecución, Bloqueado y Terminado. Las transiciones se producen como consecuencia de la admisión del proceso, la asignación o pérdida de la CPU, la solicitud o finalización de una operación de entrada y salida y la finalización de la ejecución.

4.1. Proceso admitido, pero sin memoria disponible

Mientras todos los recursos de memoria están ocupados, el proceso no puede incorporarse efectivamente a la cola de listos. En el modelo básico de cinco estados, la situación se representa manteniendo al proceso en Nuevo hasta que exista memoria suficiente.

Secuencia:

Nuevo — disponibilidad de memoria — Listo — asignación de CPU — Ejecución

La espera inicial no representa una espera por la CPU, sino una demora de admisión causada por la falta de memoria. Una vez admitido, el proceso pasa a Listo. Posteriormente, el planificador de corto plazo lo selecciona y lo lleva a Ejecución.

4.2. Solicitud de entrada y salida

Cuando el proceso en ejecución solicita una operación sobre un disco lento, no puede continuar utilizando la CPU porque debe esperar la finalización de la operación.

Secuencia:

Ejecución — solicitud de entrada y salida — Bloqueado — finalización de entrada y salida — Listo — asignación inmediata de CPU — Ejecución

El proceso pasa a Bloqueado porque espera un evento externo. Cuando el disco termina la operación, vuelve a Listo. Dado que el enunciado indica que es seleccionado inmediatamente, la última transición ocurre sin una espera adicional apreciable.

4.3. Llegada de un proceso de máxima prioridad

El sistema utiliza planificación apropiativa. Por lo tanto, la llegada de un proceso con prioridad superior provoca el desalojo del proceso que estaba utilizando la CPU.

Secuencia del proceso que estaba ejecutándose:

Ejecución — llegada de un proceso de mayor prioridad — Listo

Secuencia del nuevo proceso:

Nuevo — admisión — Listo — selección del planificador — Ejecución

El proceso desalojado no termina ni se bloquea: conserva su contexto y queda en la cola de listos para continuar más adelante.

4.4. Agotamiento del cuanto en tiempo compartido

Cuando un proceso agota su cuanto y todavía no terminó su ráfaga, pierde la CPU, pero sigue estando preparado para ejecutar.

Secuencia:

Ejecución — agotamiento del cuanto — Listo

El proceso se reencola, generalmente al final de la cola de listos, y otro proceso recibe la CPU. Si más adelante vuelve a ser seleccionado, transita nuevamente de Listo a Ejecución.

Ejercicio 5: Colas multinivel retroalimentadas

5.1. Configuración

- Cola 1: prioridad alta, Round Robin con cuanto de 7 milisegundos.
- Cola 2: prioridad media, Round Robin con cuanto de 14 milisegundos.
- Cola 3: prioridad baja, FCFS.
- Todo proceso nuevo ingresa a la Cola 1.
- Si agota el cuanto de la Cola 1, desciende a la Cola 2.
- Si agota el cuanto de la Cola 2, desciende a la Cola 3.
- Una cola inferior solo se ejecuta cuando las colas superiores están vacías.
- La llegada de un proceso a una cola superior puede desalojar a un proceso de una cola inferior.

Proceso	Llegada	Ráfaga
P1	0	15
P2	2	6
P3	5	13
P4	12	9

5.2. Resolución paso a paso

- En el tiempo 0, P1 ingresa a la Cola 1 y se ejecuta.
- En el tiempo 2 llega P2 y se incorpora al final de la Cola 1.
- En el tiempo 5 llega P3 y también ingresa a la Cola 1.
- P1 utiliza completamente su cuanto entre 0 y 7. Como todavía necesita ocho milisegundos, desciende a la Cola 2.
- P2 se ejecuta entre 7 y 12 y termina. P4 llega exactamente en el tiempo 12 y se incorpora a la Cola 1.
- P3 se ejecuta entre 12 y 19. Consume sus siete milisegundos de Cola 1 y desciende a la Cola 2 con seis milisegundos restantes.
- P4 se ejecuta entre 19 y 28 y termina antes de agotar su cuanto de siete milisegundos.
- La Cola 1 queda vacía. P1 se ejecuta en la Cola 2 entre 28 y 36 y termina.
- P3 se ejecuta en la Cola 2 entre 36 y 42 y termina.

Diagrama de Gantt:

0 — P1 — 7 — P2 — 12 — P3 — 19 — P4 — 28 — P1 — 36 — P3 — 42

5.3. Cola en la que termina cada proceso

Proceso	Recorrido	Cola en la que termina
P1	Cola 1 durante 7 ms; luego Cola 2 durante 8 ms	Cola 2
P2	Cola 1 durante 6 ms	Cola 1
P3	Cola 1 durante 7 ms; luego Cola 2 durante 6 ms	Cola 2
P4	Cola 1 durante 9 ms	Cola 1

Ningún proceso llega a la Cola 3 porque ninguno consume completamente el cuanto de 14 milisegundos de la Cola 2.

5.4. Tiempos de retorno

Proceso	Finalización	Cálculo del retorno	Retorno
P1	36	$36 - 0$	36 ms
P2	12	$12 - 2$	10 ms
P3	42	$42 - 5$	37 ms
P4	28	$28 - 12$	16 ms

Ejercicio 6:

6.1. ¿Por qué SJF es óptimo para el tiempo de espera promedio?

SJF selecciona primero la ráfaga de CPU más corta entre los procesos disponibles. De esta forma, los trabajos breves terminan rápidamente y no permanecen detrás de trabajos largos.

Para demostrar la razón de su eficiencia, considérese una situación con dos procesos disponibles: uno con una ráfaga corta y otro con una ráfaga larga. Si se ejecuta primero el proceso largo, el proceso corto debe esperar toda la duración del proceso largo. Si se ejecuta primero el proceso corto, el proceso largo espera solamente la duración de la ráfaga corta. Por lo tanto, intercambiar el orden y ejecutar primero el trabajo corto nunca aumenta la suma de los tiempos de espera.

Repitiendo este razonamiento para todos los procesos, se obtiene que SJF minimiza el tiempo de espera promedio cuando se conocen las duraciones de las ráfagas.

Su dificultad práctica consiste en que el sistema operativo normalmente no conoce de antemano cuánto tiempo necesitará un proceso en su próxima ráfaga de CPU. La duración solo puede estimarse a partir del comportamiento anterior del proceso. Además, las ráfagas pueden variar, por lo que la estimación puede ser incorrecta. También existe riesgo de inanición para los procesos largos si continúan llegando procesos cortos.

6.2. Diferencia entre los planificadores de largo y corto plazo

El **planificador de largo plazo**, también llamado planificador de trabajos, decide qué procesos son admitidos en el sistema. Su función principal es controlar la cantidad de procesos que entran a la memoria y, por lo tanto, influir directamente en el grado de multiprogramación.

El **planificador de corto plazo**, también llamado planificador de CPU, selecciona cuál de los procesos que ya están en la cola de listos obtiene el procesador. Debe actuar con mucha frecuencia y tomar decisiones rápidas.

Característica	Planificador de largo plazo	Planificador de corto plazo
Decisión principal	Admitir o no admitir trabajos	Asignar la CPU a un proceso listo
Frecuencia de ejecución	Relativamente baja	Muy alta
Cola involucrada	Trabajos o procesos nuevos	Cola de procesos listos
Impacto principal	Grado de multiprogramación	Uso de CPU, espera y respuesta

El planificador de largo plazo tiene el impacto más directo sobre el grado de multiprogramación, porque determina cuántos procesos son admitidos simultáneamente en memoria y compiten por los recursos.

6.3. Mecanismo para evitar la inanición

Un mecanismo habitual es el **envejecimiento** o *aging*. Consiste en aumentar gradualmente la prioridad de un proceso mientras permanece esperando en la cola de listos.

Por ejemplo, un proceso de baja prioridad puede comenzar con una prioridad desfavorable, pero después de esperar cierto tiempo su prioridad se incrementa. Cuando alcanza una prioridad suficientemente alta, puede ser elegido antes que procesos que llegaron posteriormente.

El envejecimiento evita que un proceso permanezca indefinidamente bloqueado en la cola de listos. De esta manera, se conserva la preferencia por los procesos importantes, pero se garantiza que los procesos de baja prioridad reciban eventualmente tiempo de CPU.

Otra alternativa consiste en utilizar realimentación entre colas: los procesos que esperan demasiado pueden ascender temporalmente a una cola de mayor prioridad. En ambos casos, el objetivo es impedir

la asignación permanente de la CPU a los mismos procesos y garantizar una forma de justicia en la planificación.